

AI ASSISTED CODING

LAB EXAM-2

Name: Shashikanth G

HT NO: 2403A51245

Batch: 11

SUBGROUP 0:

O.1 — [S18O1] Point-in-polygon (ray casting)

Context:

Geofencing checks in real estate listings platform.

Your Task:

Ray-casting point-in-polygon; edges inside.

Data & Edge Cases:

Square example; return booleans.

AI Assistance Expectation:

Crossing-count logic.

Constraints & Notes:

Return list[bool].

Sample Input

poly=[(0,0),(4,0),(4,4),(0,4)], pts=[(2,2),(5,5)]

Sample Output

[True, False]

Acceptance Criteria: Edges inside

AI GENERATED CODE:

```
def points_in_polygon(poly, pts):
    def is_inside(polygon, point):
        x, y = point
        n = len(polygon)
        inside = False
        for i in range(n):
            xi, yi = polygon[i]
            xj, yj = polygon[(i + 1) % n]
            # Check if point is on an edge
            if ((yi == yj and y == yi and min(xi, xj) <= x <= max(xi, xj)) or
                (xi == xj and x == xi and min(yi, yj) <= y <= max(yi, yj))):
                return True
            # Ray-casting
            if ((yi > y) != (yj > y)):
                x_intersect = (xj - xi) * (y - yi) / (yj - yi + 1e-12) + xi
                if x < x_intersect:
                    inside = not inside
        return inside

    return [is_inside(poly, pt) for pt in pts]

# Example usage:
poly = [(0,0),(4,0),(4,4),(0,4)]
pts = [(2,2),(5,5)]
print(points_in_polygon(poly, pts)) # Output: [True, False]
```

OUTPUT:

```
[Running] python -u "c:\Users\gshas\OneDrive\Desktop\AI AC\LAB 1\exam"
[True, False]
```

Summary

The [points_in_polygon](#) function determines if each point in [pts](#) is inside the polygon [poly](#) using the ray-casting algorithm. It returns a list of booleans, where each boolean indicates whether the corresponding point is inside (or on the edge of) the polygon.

Edge Cases Handled

- **Points on Edges:**

The function explicitly checks if a point lies exactly on a polygon edge (horizontal or vertical). If so, it returns True for that point.

- **Points Outside:**
Points outside the polygon return False.
- **Points Inside:**
Points strictly inside the polygon return True.
- **Polygon with Colinear Edges:**
The edge check covers cases where edges are colinear with the point.
- **Degenerate Cases:**
The function assumes a valid polygon and non-empty points list. It does not handle self-intersecting polygons or empty input.

O.2 — [S18O2] Compute rolling median (w=3)

Context:

Rolling medians (w=3) for real estate listings platform anomaly detection.

Your Task:

Return median per sliding window; efficient approach.

Data & Edge Cases:

Example list provided; large-n acceptable.

AI Assistance Expectation:

AI to propose bisect/heaps and tests.

Constraints & Notes:

Include small edge tests.

Sample Input

[1, 3, 2, 5, 4]

Sample Output

[2, 3, 4]

Acceptance Criteria: Efficient and correct.

AI-GENERATED CODE:

```
LAB 1 > exam2 > ...
1 import bisect
2
3 def rolling_median(nums, w=3):
4     if w > len(nums) or w == 0:
5         return []
6     window = sorted(nums[:w])
7     medians = [window[w // 2]]
8     for i in range(w, len(nums)):
9         # Remove outgoing element
10        out = nums[i - w]
11        idx = bisect.bisect_left(window, out)
12        window.pop(idx)
13        # Insert incoming element
14        bisect.insort(window, nums[i])
15        medians.append(window[w // 2])
16    return medians
17
18 # Sample input
19 print(rolling_median([1, 3, 2, 5, 4])) # Output: [2, 3, 4]
20
21 # Edge tests
22 print(rolling_median([1, 2], 3))      # Output: []
23 print(rolling_median([5, 5, 5, 5], 3)) # Output: [5, 5]
24 print(rolling_median([], 3))          # Output: []
25 print(rolling_median([1, 2, 3], 1))   # Output: [1, 2, 3]
```

OUTPUT:

```
[Running] python -u "c:\Users\gshas\OneDrive\Desktop\AI AC\LAB 1\exam2"
[2, 3, 4]
[]
[5, 5]
[]
[1, 2, 3]

[Done] exited with code=0 in 0.083 seconds
```

SUMMARY

The [rolling_median](#) function efficiently computes the median for each sliding window of size [w](#) in a list using the [bisect](#) module for fast insertions and removals. It returns a list of medians for each window position.

Edge Cases Handled

- **Window Larger Than List or Zero:**
Returns [] if [w > len\(nums\)](#) or [w == 0](#).
- **All Elements Same:**
Handles repeated values correctly (e.g., [5, 5, 5, 5] with [w=3](#) returns [5, 5]).
- **Empty List:**
Returns [] for empty input.

- **Window Size One:**
Returns the original list (each element is its own median).
- **General Case:**
Correctly computes medians for arbitrary lists and window sizes.